

DevOps — это подход, объединяющий людей, процессы и технологии.

- DevOps — это методология, которая объединяет разработку (Dev) и эксплуатацию (Ops) в единый, согласованный процесс;
- DevOps в первую очередь затрагивает культурные изменения в работе команды, и только затем — технические инструменты для их реализации (Git или CI/CD);
- Git и другие инструменты — часть культурной трансформации, и почему правильное использование Git помогает улучшить работу над кодом и взаимодействие в команде.

В 2009 году бельгийский инженер Патрик Дебуа (Patrick Debois) организовал первую конференцию DevOpsDays в Генте, Бельгия. Дебуа, работая консультантом на многих проектах в разных компаниях, столкнулся с проблемой: как сделать так, чтобы разработка и эксплуатация действительно работали вместе. Вдохновившись докладом инженеров из Flickr о коллаборации Dev и Ops, он решил объединить людей, которым близка эта идея. Так и появилось сообщество DevOps, и постепенно такой подход к управлению проектами превратился в глобальное движение, выйдя далеко за рамки инструментов — в область культуры, процессов и управления командами. DevOps предлагает решение проблем традиционной разработки: выравнивание окружений, совместная ответственность за релиз, инфраструктура как код, автоматизированное тестирование миграций и повышенная прозрачность. Всё это помогает избежать сценариев «а у нас работало» с любой стороны стены — DevOps её разрушает и вместо передачи мяча вводит совместную работу. Возникает единая команда, где знание и ответственность распределяются, а процессы становятся прозрачными и предсказуемыми. Основой DevOps можно назвать культуру сотрудничества, автоматизации и постоянного улучшения, про которую мы и поговорим далее.

Случай

Пятничный вечер, команда завершила крупную фичу: новая схема БД и фоновый обработчик. Релиз помечен готовым, выкладка запланирована на полночь с субботы на воскресенье. После запуска миграции выясняется:

- На тестовой среде миграция занимала 20 секунд — на проде блокирует критические таблицы на 40 минут.
- Новый обработчик не может подключиться к базе: в продакшене другая схема DNS-имён для внутренних сервисов.
- Разработчики на выходных, трубку не берут.

Администратор пытается откатиться, но инструкция устарела и не учитывает новые колонки. Сайт лежит в пик посещаемости.

Какую DevOps-практику стоит внедрить, чтобы не допустить повторения?

IaC делает окружения идентичными, а культура совместной ответственности подразумевает, что Ops участвует в ревью архитектуры до деплоя.

Принципы CALMS

Мы рассмотрели принципы DevOps. Теперь перейдём к более детальному изучению модели CALMS, которая объединяет пять ключевых принципов, лежащих в основе зрелого DevOps-подхода и помогающих организациям развивать и измерять свои процессы.

Culture — культура

DevOps невозможен без культуры доверия и сотрудничества. Показатель зрелости команды — то, как она реагирует на инциденты: ищет виноватого или меняет систему так, чтобы больше не было повода для обвинений. Все члены команды должны знать, зачем они делают свою работу и как она влияет на конечный результат. Ретроспективы, постмортемы, автоматизация, обсуждение метрик — всё это часть культуры вокруг систем, которые не ломаются. А если и ломаются, то быстро восстанавливаются и становятся лучше.

Плохой пример: после инцидента начинается поиск «виноватого». Один человек публично обвиняется в ошибке, остальные предпочитают молчать и перестраховываться.

Хороший пример: проводится расследование проблемы без обвинений, собираются данные, документируются выводы, принимаются меры по предотвращению повторения ситуации.

Automation — автоматизация

Ручные операции рано или поздно приводят к ошибкам — даже у внимательного инженера. Поэтому в DevOps необходима автоматизация: она делает процессы предсказуемыми, повторяемыми и прозрачными, обеспечивает контроль и аудит, а команда при этом освобождается для более сложных задач.

Автоматизировать можно практически всё: сборку и тестирование через CI, развёртывание через CD, управление инфраструктурой с помощью IaC, мониторинг и алерты — и это далеко не полный список.

Плохой пример: каждый релиз сопровождается ручными действиями по инструкции из Wiki, которую никто давно не обновлял. Один сотрудник знает, как починить систему в случае сбоя.

Хороший пример: весь процесс сборки, тестирования и деплоя описан в скриптах автоматизации. Любой инженер может нажать кнопку — и всё заработает одинаково в тестовой среде и в продакшне.

Lean — бережливость

DevOps опирается на идеи бережливого производства: убираем потери, не делаем лишних действий и как можно быстрее даём пользователю ценность. Отсюда подход с MVP и быстрой обратной связью, а не «год делаем идеал, потом показываем».

Плохой пример: команда восемь месяцев пишет сложную архитектуру «на вырост», которую невозможно протестировать частями. Все фичи связаны, и релиз можно сделать только один — в конце.

Хороший пример: MVP запускается через месяц, команда собирает отзывы пользователей, адаптирует архитектуру под реальные потребности и планирует релизы инкрементально.

Measurement — измерения

«Ты не можешь улучшить то, что не измеряешь». DevOps поощряет сбор метрик: количество деплоев, время восстановления после сбоя, процент неуспешных релизов, скорость обратной связи. На основе этих данных можно выявлять узкие места и улучшать процессы.

Плохой пример: команда не знает, сколько у неё фейлов в автоматизации, не отслеживает время восстановления после падений, ориентируется на ощущения и память.

Хороший пример: в дашборде отображаются ключевые показатели: время восстановления после падения, время сборки, количество аварийных откатов. Они используются для принятия решений и обсуждаются на ретроспективах.

Sharing — обмен знаниями

Документация, ретроспективы, внутренние Wiki, обучение новых сотрудников — всё это обеспечивает распространение знаний. Команда не должна зависеть от одного человека, и знания не должны прятаться в головах. Git как система контроля версий — тоже инструмент Sharing: он хранит историю изменений и делает работу прозрачной.

Плохой пример: важные команды деплоя или скрипты лежат на рабочем столе одного инженера. Вся настройка окружения хранится в его голове.

Хороший пример: инструкции по запуску и восстановлению задокументированы. Всё конфигурируется через Git. Новые сотрудники могут освоиться без постоянных вопросов к старшим коллегам.

Почему именно CALMS?

Принципы **CALMS** охватывают не только технические аспекты, но и культурные, управленческие и организационные. Они выбраны потому, что DevOps переосмысляет работу команды, процесс принятия решений и повышение устойчивости к изменениям.

- **Culture** решает проблемы изоляции и недоверия между Dev и Ops;
- **Automation** позволяет избавиться от ручного труда и добиться повторяемости;
- **Lean** помогает фокусироваться на ценности, а не на процессе ради процесса;
- **Measurement** даёт факты, а не предположения, и позволяет управлять на основе данных;
- **Sharing** формирует устойчивую и гибкую команду, не зависящую от отдельных людей.

Если выпадет хоть один компонент, система будет либо хрупкой, либо неэффективной.

Measurement